

GUÍA INICIAL MANEJO DE ARCHIVOS TXT

Primeros pasos con archivos TXT en Python

Crear · agregar · leer · mostrar

Propósito: Aprender únicamente el ciclo básico de un archivo de texto. Por ahora no trabajaremos con diccionarios, búsquedas, eliminaciones ni registros complejos.

1. ¿Qué aprenderemos primero?

Antes de construir un sistema de producción o de ventas, necesitamos entender cómo un programa guarda información para utilizarla después. En esta guía trabajaremos con un archivo llamado registro.txt y realizaremos solo cuatro acciones:

- Crear una carpeta y una ruta de archivo.
- Crear el archivo si todavía no existe.
- Agregar información al final sin borrar lo anterior.
- Leer cada línea y mostrarla en pantalla.

Idea clave: El archivo queda guardado en el computador. Si cerramos el programa y lo ejecutamos nuevamente, la información anterior seguirá allí.

2. Preparar el proyecto

- Crea una carpeta para esta actividad.
- Abre esa carpeta en Visual Studio Code.
- Crea un archivo llamado ejercicio_archivos.py.
- No crees manualmente el archivo TXT: debe crearlo Python.

2.1. La ruta que usaremos

datos/registro.txt

La carpeta datos estará dentro de la carpeta del proyecto. Esta es una ruta relativa: no depende de una dirección específica de Windows o macOS.

Todavía no agregues datos: En el primer ejercicio solo prepararemos la ubicación del archivo. La información se agregará en los ejercicios siguientes.

3. Ejercicios paso a paso

Completa un ejercicio, ejecútalo y verifica el resultado antes de continuar. No avances si no puedes explicar qué ocurrió.

Ejercicio 1. Crear la carpeta y la ruta

Objetivo: Reconocer que un archivo tiene una ubicación.

Realiza: Importa Path. Crea una constante para la carpeta datos y otra para el archivo registro.txt.

Pista: Usa Path("datos") y luego combina la carpeta con / "registro.txt".

Verifica: Imprime ARCHIVO y observa que la ruta corresponde a datos/registro.txt.

```
from pathlib import Path

CARPETA = Path("datos")
ARCHIVO = CARPETA / "registro.txt"

print(ARCHIVO)
```

Ejercicio 2. Crear la carpeta si no existe

Objetivo: Evitar errores cuando la carpeta aún no está creada.

Realiza: Agrega una instrucción que cree la carpeta datos desde Python.

Pista: La instrucción mkdir(exist_ok=True) crea la carpeta y no genera error si ya existe.

Verifica: Ejecuta el programa y verifica que aparezca la carpeta datos.

```
CARPETA.mkdir(exist_ok=True)
```

Ejercicio 3. Crear el archivo TXT

Objetivo: Comprender que el modo a crea el archivo cuando no existe.

Realiza: Abre ARCHIVO usando with open(...) y el modo a. Todavía no escribas información.

Pista: El modo a significa append, es decir, agregar. También crea el archivo si no existe.

Verifica: Debe aparecer datos/registro.txt. Al ejecutar nuevamente, no debe borrarse.

```
with open(ARCHIVO, "a", encoding="utf-8") as archivo:
    pass
```

Observa: La instrucción pass significa que todavía no escribiremos. El archivo se crea de todas formas.

Ejercicio 4. Escribir una primera línea

Objetivo: Guardar información en el archivo.

Realiza: Guarda una frase sencilla, por ejemplo: Primera prueba de archivo. Ejecuta el programa y revisa el TXT.

Pista: Usa una variable y una f-string para armar la línea: `f"{texto}\n"`. El salto de línea deja cada registro separado.

Verifica: El archivo debe contener exactamente la frase escrita.

```
texto = "Primera prueba de archivo"

with open(ARCHIVO, "a", encoding="utf-8") as archivo:
    archivo.write(f"{texto}\n")
```

Ejercicio 5. Ejecutar nuevamente y agregar otra línea

Objetivo: Comprobar la diferencia entre agregar y reemplazar.

Realiza: Vuelve a ejecutar el programa, pero cambia la frase. Revisa el archivo después de la segunda ejecución.

Pista: Mantén el modo a. No uses w, porque w reemplazaría lo que ya existe.

Verifica: El TXT debe conservar la primera frase y mostrar la segunda debajo.

4. Agregar información ingresada por el usuario

Ejercicio 6. Solicitar un dato con input()

Objetivo: Conectar una entrada del usuario con una escritura en archivo.

Realiza: Solicita un nombre y guarda un saludo en el archivo, por ejemplo: Hola, Ana.

Pista: Guarda la respuesta en una variable y arma la línea con `f"Hola, {nombre}\n"`.

Verifica: Cada ejecución debe agregar un nuevo saludo sin borrar los anteriores.

```
nombre = input("Ingrese su nombre: ").strip()

with open(ARCHIVO, "a", encoding="utf-8") as archivo:
    archivo.write(f"Hola, {nombre}\n")
```

Ejercicio 7. Registrar un producto

Objetivo: Agregar datos útiles al caso de producción.

Realiza: Solicita el nombre de un producto y guárdalo en una línea. Por ahora guarda solo el nombre.

Pista: Todavía no uses diccionarios ni separadores.

Verifica: Después de registrar tres productos, el archivo debe mostrar tres líneas nuevas.

```
producto = input("Ingrese un producto: ").strip()

with open(ARCHIVO, "a", encoding="utf-8") as archivo:
    archivo.write(f"{producto}\n")
```

Ejercicio 8. Agregar varios datos con un ciclo

Objetivo: Utilizar un ciclo para repetir una operación.

Realiza: Usa un ciclo for para solicitar tres productos y agregarlos al mismo archivo.

Pista: Mantén el modo a y termina cada escritura con `\n`.

Verifica: El archivo debe conservar los datos anteriores y sumar tres productos nuevos.

```
for i in range(3):
    producto = input("Ingrese un producto: ").strip()
    with open(ARCHIVO, "a", encoding="utf-8") as archivo:
        archivo.write(f"{producto}\n")
```

Pausa de comprensión: Explica con tus palabras por qué usamos el modo a y qué ocurriría si utilizáramos w.

5. Leer y mostrar el archivo

Ejercicio 9. Leer todas las líneas

Objetivo: Recuperar la información que quedó guardada.

Realiza: Abre el archivo en modo r y recórrelo con un ciclo for. Imprime cada línea.

Pista: El modo r significa read. Usa `linea.strip()` para quitar el salto de línea.

Verifica: El programa debe mostrar en pantalla todas las líneas guardadas en el TXT.

```
with open(ARCHIVO, "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea.strip())
```

Ejercicio 10. Separar escritura y lectura en funciones

Objetivo: Comprender que una función puede reunir una tarea completa.

Realiza: Crea `agregar_dato()` para solicitar y guardar una línea. Crea `leer_datos()` para leer y mostrar las líneas.

Pista: Cada función debe tener una responsabilidad: una escribe y la otra lee.

Verifica: Desde el programa principal puedes llamar a una función y después a la otra.

```
def agregar_dato():
    # solicitar y guardar una línea
    pass

def leer_datos():
    # abrir, recorrer y mostrar líneas
    pass
```

Ejercicio 11. Crear un menú mínimo

Objetivo: Usar las funciones sin mezclar todas las instrucciones.

Realiza: Construye un menú con tres opciones: 1) agregar un dato, 2) leer y mostrar datos, 3) salir.

Pista: Usa `while True`, `input()`, `if/elif` y `break`. No agregues eliminación ni búsqueda todavía.

Verifica: Puedes agregar varias líneas, leerlas y volver al menú sin cerrar el programa.

6. Código base de referencia

Este código muestra la estructura mínima. No lo copies sin analizar qué hace cada línea.

```
from pathlib import Path

CARPETA = Path("datos")
ARCHIVO = CARPETA / "registro.txt"

def preparar_archivo():
    CARPETA.mkdir(exist_ok=True)
    with open(ARCHIVO, "a", encoding="utf-8"):
        pass

def agregar_dato():
    dato = input("Ingrese un dato: ").strip()
    with open(ARCHIVO, "a", encoding="utf-8") as archivo:
        archivo.write(f"{dato}\n")

def leer_datos():
    with open(ARCHIVO, "r", encoding="utf-8") as archivo:
        for linea in archivo:
            print(linea.strip())

preparar_archivo()
agregar_dato()
leer_datos()
```

Qué debes entender: Primero se prepara la ruta, después se agrega una línea, luego se vuelve a abrir el mismo archivo para leerlo y mostrarlo.

7. Errores frecuentes

- Usar w para agregar: borra el contenido anterior.
- Olvidar el salto de línea: todas las entradas quedarían pegadas.
- Leer con r antes de crear el archivo: puede producir FileNotFoundError.
- No utilizar with: el archivo podría no cerrarse correctamente.
- Crear manualmente el TXT: en esta guía debe crearlo Python.
- Intentar resolver todavía ventas, diccionarios o eliminaciones: esas operaciones vendrán después.

8. Checklist de logro

- Puedo explicar qué representa la ruta datos/registro.txt.

- Puedo crear una carpeta desde Python.
- Puedo crear un TXT con modo a.
- Puedo agregar una línea sin borrar las anteriores.
- Puedo leer el archivo con modo r.
- Puedo recorrerlo con un ciclo for.
- Puedo mostrar cada línea con print().
- Puedo separar la escritura y la lectura en funciones.

Lo que viene después: Cuando este flujo esté claro, podremos agregar registros con varios campos, usar diccionarios y finalmente trabajar con el caso de producción o ventas.